

UNIVERSITÀ DEGLI STUDI DI MILANO  
Facoltà di Scienze e Tecnologie  
Dipartimento di Informatica “Giovanni Degli Antoni”



---

## Come Rust previene i problemi di sicurezza della memoria

---

*Elaborato di:*  
Marco Agnoli  
Matricola XXXXX

Corso di Tecniche di Protezione del Software  
Anno Accademico 2025/2026



# Indice

|          |                                                                                                                         |           |
|----------|-------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                                                                                     | <b>1</b>  |
| <b>2</b> | <b>Temporal safety: ownership e borrowing</b>                                                                           | <b>3</b>  |
| 2.1      | Ownership . . . . .                                                                                                     | 3         |
| 2.2      | Tipi <i>Copy</i> e tipi <i>non-Copy</i> . . . . .                                                                       | 4         |
| 2.3      | Borrowing e riferimenti mutabili . . . . .                                                                              | 5         |
| 2.4      | Il borrow checker . . . . .                                                                                             | 6         |
| 2.4.1    | I riferimenti non devono sopravvivere ai valori referenziati                                                            | 6         |
| 2.4.2    | Non deve esistere aliasing pericoloso . . . . .                                                                         | 7         |
| 2.4.3    | Il trasferimento di ownership non deve lasciare accessi non<br>validi . . . . .                                         | 9         |
| 2.5      | Ownership condivisa: <code>Rc&lt;T&gt;</code> e <code>Arc&lt;T&gt;</code> . . . . .                                     | 10        |
| 2.6      | Interior mutability: <code>Cell&lt;T&gt;</code> , <code>RefCell&lt;T&gt;</code> e <code>Mutex&lt;T&gt;</code> . . . . . | 11        |
| <b>3</b> | <b>La protezione degli attacchi: use-after-free, double free</b>                                                        | <b>13</b> |
| 3.1      | Use-after-free nello heap . . . . .                                                                                     | 13        |
| 3.2      | Use-after-free nello stack . . . . .                                                                                    | 14        |
| 3.3      | Double free . . . . .                                                                                                   | 15        |
| <b>4</b> | <b>Spatial safety e attacchi di buffer overflow</b>                                                                     | <b>16</b> |
| 4.1      | Nessuna deferenzaiazione né aritmetica dei puntatori . . . . .                                                          | 16        |
| 4.2      | Slices e wide pointers . . . . .                                                                                        | 17        |
| 4.3      | Bounds checking statico e dinamico . . . . .                                                                            | 18        |
| <b>5</b> | <b>Riassumendo: Rust e gli attacchi visti a lezione</b>                                                                 | <b>21</b> |
| <b>6</b> | <b>I limiti di Rust</b>                                                                                                 | <b>23</b> |
| 6.1      | Unsafe Rust . . . . .                                                                                                   | 23        |
| 6.2      | Uso di <code>Rc</code> e <code>RefCell</code> e riferimenti circolari . . . . .                                         | 25        |
| 6.3      | Il compilatore <code>rustc</code> e le opzioni per la gestione della memoria . .                                        | 27        |
| <b>7</b> | <b>Prestazioni</b>                                                                                                      | <b>28</b> |
| 7.1      | Rust non usa un garbage collector . . . . .                                                                             | 28        |
| 7.2      | Confronto con C e C++ . . . . .                                                                                         | 28        |
| 7.3      | Il costo del bounds checking: un esperimento . . . . .                                                                  | 29        |

|           |                                                                              |           |
|-----------|------------------------------------------------------------------------------|-----------|
| <b>8</b>  | <b>Il database Juliet e il suo utilizzo nello studio della memory safety</b> | <b>31</b> |
| 8.1       | Metodologia sperimentale . . . . .                                           | 31        |
| 8.2       | Risultati osservati . . . . .                                                | 32        |
| 8.3       | Insegnamenti dalla parte sperimentale . . . . .                              | 34        |
| <b>9</b>  | <b>Uso di Rust fuori dall'ambito accademico: Android, Microsoft e Linux</b>  | <b>35</b> |
| 9.1       | Android . . . . .                                                            | 35        |
| 9.2       | Microsoft e il porting della libreria crittografica SymCrypt . . . .         | 36        |
| 9.3       | Rust nel kernel Linux . . . . .                                              | 37        |
| <b>10</b> | <b>Conclusioni</b>                                                           | <b>38</b> |
|           | <b>Bibliografia</b>                                                          | <b>40</b> |

# Capitolo 1

## Introduzione

La sicurezza del software non può essere ridotta a firewall, antivirus o controlli del sistema operativo. Come discusso durante le lezioni del corso, questi strumenti agiscono spesso come difese esterne ma se il codice delle applicazioni o dei sistemi che vengono utilizzati contiene vulnerabilità (di memoria o di altro tipo), un attaccante può sfruttarle ben prima che le difese perimetrali riescano a fermarlo.

Le vulnerabilità di *memory safety* sono storicamente tra le più importanti nella sicurezza dei sistemi – e anche oggi, ad esempio in Chrome, circa il 70 % degli *high-severity bugs* è dovuto a problemi di sicurezza della memoria. Un *buffer overflow* permette di scrivere oltre i limiti di un buffer; un *use-after-free* (UAF) permette di dereferenziare un puntatore verso memoria già liberata in precedenza; un *dangling pointer* continua a riferirsi a un oggetto che in memoria non esiste più. Queste classi di bug sono state al centro di exploit celebri e hanno motivato decenni di mitigazioni, dallo *stack canary* all'*Address Space Layout Randomization* (ASLR), fino alla *Data Execution Prevention* (DEP).

La memory safety è scomposta in due proprietà fondamentali: *spatial safety* e *temporal safety*. La prima richiede che un puntatore possa accedere solo alla memoria entro i limiti dell'oggetto a cui appartiene; la seconda richiede che la memoria referenziata sia ancora viva, cioè allocata e valida. Molti attacchi classici – tra cui quelli studiati nel corso – violano una o entrambe queste proprietà. Sempre durante il corso abbiamo studiato come si sia cercato negli anni di introdurre nuovi costrutti per rendere memory safe i linguaggi come C e C++, con i quali si sono sviluppati molti software che ancora oggi sono di importanza vitale. Abbiamo anche studiato che linguaggi più moderni, da Java a Python a C#, con l'utilizzo di *garbage collector*, abbiano una gestione migliore della sicurezza della memoria. Abbiamo però visto come queste correzioni imponessero di pagare un prezzo alto in termini di performance. Quindi, dove le performance sono importanti, il problema della gestione sicura della memoria permane.

Rust nasce nel 2015 precisamente per affrontare questo problema. Il linguaggio, inizialmente sviluppato da Mozilla Research e oggi supportato dalla *Rust Foundation*, si propone di offrire controllo a basso livello e prestazioni da linguaggio di sistema, ma con forti garanzie di sicurezza della memoria già in fase di compilazio-

ne [1, 2, 3, 4]. La sua idea di fondo è spostare quanto più possibile il costo della sicurezza dal *runtime* al *compile-time*, tramite un linguaggio fortemente tipizzato, l'*ownership*, lo *scope* e altri meccanismi che vedremo.

Quello che vedremo in questo documento è come Rust affronta e risolve i problemi di memory safety ma anche quali sono i suoi limiti. Vedremo anche il livello di adozione del linguaggio e come questo venga usato in contesti importanti dell'industria del software.

## Capitolo 2

# Temporal safety: ownership e borrowing

## 2.1 Ownership

In Rust ogni valore ha un proprietario attivo, detto *owner*. Il compilatore è responsabile della distruzione del valore quando l'owner esce dal suo *scope* – e se il valore è allocato nello heap, la memoria viene automaticamente deallocata. Questo compito non è lasciato alla disciplina del programmatore come in C ma viene verificato staticamente dal compilatore. Dal punto di vista della sicurezza del software, tale impostazione introduce una forte garanzia di temporal safety, cioè la certezza che l'accesso alla memoria sia sempre coerente con il ciclo di vita dell'oggetto a cui si riferisce.

La differenza rispetto ai linguaggi tradizionali emerge chiaramente osservando il comportamento delle assegnazioni. In C/C++ è possibile creare alias di puntatori che riferiscono alla stessa area di memoria heap:

```
int main() {
    char *s = malloc(5); // alloca 5 byte sull'heap per contenere la stringa "ciao"
    strcpy(s, "ciao");   // copia la stringa "ciao" nella memoria puntata da s
    char *t = s;         // copia il puntatore: t ora punta alla stessa memoria di s
    printf("%s\n", t);   // stampa la stringa tramite il puntatore t
    printf("%s\n", s);   // stampa la stessa stringa tramite il puntatore s
}
```

Figura 2.1: Alias di puntatori in C.

In questo esempio entrambe le variabili fanno riferimento alla stessa allocazione. Il linguaggio non impone alcuna regola su quale puntatore debba liberare la memoria. Se il programma eseguisse una **free** tramite uno dei due alias, l'altro continuerebbe a contenere l'indirizzo della memoria ormai non più valida. Questa situazione rappresenta una delle condizioni tipiche da cui derivano vulnerabilità come *use-after-free* o *double free*, spesso sfruttabili per la corruzione dello heap e il controllo del flusso di esecuzione.

Rust adotta un approccio diverso: quando un valore nello heap viene assegnato a una nuova variabile, non viene copiato il puntatore ma viene trasferita l'*ownership*.

```
fn main() {
    let s = String::from("ciao"); // alloca una String sull'heap e assegna ownership a s
    let t = s;                    // trasferisce l'ownership della stringa da s a t
    println!("{}", t);           // uso corretto: t è ora il proprietario della stringa
    println!("{}", s);           // errore di compilazione: s non è più valida dopo il move
}
```

Figura 2.2: Trasferimento di ownership in Rust.

Dopo l'assegnazione `let t = s`; la variabile originale `s` non può più essere utilizzata. Il compilatore impedisce quindi che due variabili possano considerarsi contemporaneamente responsabili della stessa risorsa. Questo semplice meccanismo elimina strutturalmente una parte significativa delle condizioni necessarie allo sfruttamento delle vulnerabilità di memoria.

## 2.2 Tipi *Copy* e tipi *non-Copy*

È importante fin da subito comprendere la distinzione tra tipi *Copy* e tipi che gestiscono memoria dinamica, detti *non-Copy*. I tipi semplici, come gli interi, vengono copiati automaticamente perché il loro valore è contenuto interamente nello stack.

```
fn main() {
    let a = 10; // i32 implementa Copy
    let b = a;  // viene copiata una copia del valore
    println!("{}", a); // OK: a è ancora valido
    println!("{}", b); // OK: b contiene la copia
}
```

Figura 2.3: Tipi *Copy*: copia di valore.

In questo caso non esiste una risorsa condivisa nello heap e quindi non si crea alcun rischio di corruzione della memoria.

Per i tipi *heap-allocated*, invece, Rust utilizza il trasferimento di ownership visto in precedenza proprio per evitare che più variabili possano tentare di gestire lo stesso spazio di memoria.

```
fn main() {
    let a = vec![1,2,3]; // Vec è nello heap e NON è Copy
    let b = a;           // ownership trasferita (move)
    println!("{:?}", a); // ERRORE: a è stato mosso
    println!("{:?}", b); // OK: b è il nuovo proprietario
}
```

Figura 2.4: Move di un valore *non-Copy*.

In questo caso quindi, dopo l'assegnazione `let b = a;`, l'ownership del valore rappresentato con il vettore dinamico `vec![1,2,3]` passa da `a` a `b`, e tutto ciò che si riferisce ad `a` viene reso inutilizzabile e causa di errore di compilazione.

## 2.3 Borrowing e riferimenti mutabili

Il *borrowing* in Rust offre la possibilità di permettere l'uso temporaneo di un valore con un riferimento ma senza trasferirne l'ownership. Nella pratica un programma non vuole sempre cambiare il valore di una risorsa: molto spesso vuole soltanto leggerla o modificarla temporaneamente, mantenendo però altrove la responsabilità principale del suo ciclo di vita.

Esistono due tipi di riferimenti: *mutabile* e *immutabile*. Un riferimento si dice immutabile se permette solamente di accedere al dato in lettura. Un riferimento si dice mutabile se permette anche di modificarlo. In questo caso però si richiede una condizione più forte rispetto a un riferimento immutabile: si richiede esclusività. Mentre possono esistere più riferimenti immutabili allo stesso valore, perché più letture simultanee non creano di per sé un conflitto, un riferimento mutabile presuppone che nessun altro stia usando quel valore nello stesso momento in modo incompatibile. Da qui deriva la formula sintetica spesso citata nei testi su Rust: *aliasing XOR mutability*. Significa che o esistono alias multipli in sola lettura, oppure esiste un solo accesso in scrittura, ma non entrambe le cose insieme.

Vediamo questo ulteriore esempio:

```
fn main() {
    let mut x = 10;           // variabile mutabile
    let r = &mut x;           // ref mutabile (prestito esclusivo)
    x = 20;                   // ERRORE: x è già preso in prestito
    println!("{}", r);        // uso del riferimento
}
```

Figura 2.5: Un solo riferimento mutabile alla volta.

`x` viene dichiarata come variabile mutabile, e `r` è un suo riferimento mutabile. Fin qui nulla di errato. Nel momento in cui però cerco di modificare `x` direttamente, il compilatore segnala un errore: quando `r` prende in prestito `x` come riferimento mutabile, ottiene un accesso esclusivo al valore. Finché questo prestito è attivo, non è consentito accedere a `x` in nessun altro modo (né in lettura né in scrittura). `x` torna utilizzabile solo quando il riferimento `r` non è più in uso (alla fine del suo *lifetime*, come vedremo poco più avanti).

Questa regola è una restrizione strutturale che serve a escludere una classe di errori molto comune nei linguaggi tradizionali, in particolare le *data race* e, più in generale, situazioni di aliasing non sicuro. Se infatti più riferimenti potessero accedere allo stesso dato e almeno uno di essi potesse modificarlo mentre un altro lo legge o lo usa assumendo che sia invariato, il programma entrerebbe in uno stato potenzialmente incoerente e, in alcuni contesti, anche pericoloso dal punto di vista della sicurezza.

## 2.4 Il borrow checker

Il *borrow checker* è un componente del compilatore di Rust e verifica tre proprietà fondamentali:

1. che i riferimenti non sopravvivano ai valori a cui puntano;
2. che non si verifichi *dangerous aliasing*;
3. che il trasferimento di ownership non lasci nel programma accessi non più validi.

### 2.4.1 I riferimenti non devono sopravvivere ai valori referenziati

Il primo controllo effettuato dal borrow checker riguarda la durata dei riferimenti (*lifetime*). In Rust lo *scope* definisce la visibilità sintattica di una variabile, mentre il *lifetime* rappresenta la durata durante la quale un riferimento è valido. Il principio è semplice da formulare ma molto forte nelle conseguenze: un riferimento non deve avere un *lifetime* più lungo del valore a cui si riferisce. Se questa condizione venisse violata, il riferimento finirebbe per puntare a un oggetto che non esiste più, generando un cosiddetto *dangling pointer*, puntatore pendente. Nei linguaggi con gestione manuale della memoria questo problema è estremamente comune, perché nulla impedisce di conservare l'indirizzo di una variabile oltre il suo *scope* di validità. L'esempio in C qui sotto rende bene la situazione.

```

int main() {
    int *p;           // dichiara un puntatore che vivrà anche fuori dal blocco
    {
        int x = 42;    // x vive solo all'interno di questo blocco
        p = &x;        // p riceve l'indirizzo di x
    }                // qui x esce dallo scope e viene distrutto
    printf("%d\n", *p); // dangling pointer: p punta a una variabile non più esistente
}

```

Figura 2.6: Dangling pointer in C.

Qui il problema è evidente: `x` viene distrutta alla fine del blocco interno, ma `p` conserva il suo indirizzo. L'ultima istruzione dereferenzia quindi un puntatore verso una zona di memoria che non corrisponde più a un oggetto valido. Questa è una violazione classica della temporal safety.

In Rust una costruzione analoga viene bloccata dal compilatore per il principio sopra enunciato:

```

fn main() {
    let r; // dichiara una variabile che conterrà un riferimento
    {
        let x = 42; // x vive solo all'interno di questo blocco
        r = &x; // prova ad assegnare a r un riferimento a x
    } // qui x esce dallo scope e viene distrutto
    println!("{}", r); // errore di compilazione: r vivrebbe più a lungo di x
}

```

Figura 2.7: Violazione di lifetime rifiutata dal compilatore.

Il punto cruciale è che il compilatore confronta la durata di `r` con quella di `x` e conclude che il programma non è sicuro. `r`, infatti, continuerebbe a esistere dopo la distruzione di `x`. Se Rust accettasse questo codice, introdurrebbe esattamente quel tipo di accesso a memoria non valida che il linguaggio vuole escludere nel codice *safe*. Per questo il programma viene rifiutato prima ancora di poter essere eseguito.

## 2.4.2 Non deve esistere aliasing pericoloso

Vediamo ora il secondo controllo del borrow checker: non deve esistere *dangerous aliasing* o aliasing pericoloso. Con questa espressione si intende proprio la situazione in cui esistono più riferimenti allo stesso dato mentre almeno uno di essi può modificarlo. In C e C++ una simile situazione è più che possibile, perché nulla impedisce di creare più puntatori allo stesso indirizzo e di usare uno per scrivere e l'altro per leggere.

```
int main() {
    int x = 5;           // dichiara una variabile intera nello stack e inizializza il valore a 5
    int *a = &x;         // crea un puntatore a che contiene l'indirizzo di x
    int *b = &x;         // crea un secondo puntatore b che punta allo stesso indirizzo di x
    *b = 10;             // modifica il valore di x tramite il puntatore b
    printf("%d\n", *a); // legge il valore tramite il puntatore a (stampa 10)
}
```

Figura 2.8: Aliasing tra lettura e scrittura in C.

In questo esempio il programma crea due puntatori allo stesso dato. Il puntatore `b` modifica il contenuto di `x`, mentre `a` continua a leggerlo. In un esempio così piccolo il risultato può sembrare innocuo, ma il punto teorico è che il linguaggio non impone nessun vincolo sulla coerenza di questi accessi. In programmi più complessi, o peggio ancora in presenza di concorrenza, questa libertà può diventare la base di bug di *data race*.

Rust sceglie una strada opposta: tratta come errore la coesistenza di un prestito immutabile e di un prestito mutabile sullo stesso dato, se il loro intervallo di validità si sovrappone.

```
fn main() {
    let mut x = 5; // x mutabile
    let a = &x;    // borrow immutabile
    let b = &mut x; // ERRORE: x già preso in prestito da a
    *b = 10;       // non raggiungibile
    println!("{}", a); // a usa ancora il borrow
}
```

Figura 2.9: Aliasing pericoloso rifiutato da Rust.

L'esempio sopra è molto simile a quello visto in precedenza quando abbiamo introdotto il borrowing, ma viene riproposto qui per maggiore chiarezza. Qui il compilatore rifiuta il programma perché `x` è già preso in prestito in modo immutabile da `a` quando si tenta di creare `b` come riferimento mutabile. Ciò serve a impedire che uno stesso dato sia contemporaneamente osservato come condiviso e manipolato come esclusivo. In altre parole, il borrow checker protegge il programma dall'aliasing pericoloso imponendo che lettura condivisa e scrittura esclusiva non si sovrappongano.

Non significa che Rust proibisca ogni forma di aliasing. Più riferimenti immutabili sono ammessi. Quello che viene proibito è soltanto l'aliasing pericoloso, cioè quello che, combinato con la mutabilità, potrebbe produrre inconsistenza negli accessi.

Vale la pena osservare che questa logica si estende anche alla distruzione di un valore. Chiamare `drop` su un dato è concettualmente un'operazione esclusiva su di esso – non dissimile da un accesso mutabile – perché consuma l'ownership in modo irreversibile - `drop` è l'istruzione Rust equivalente a `free` in C. Di conseguenza, il compilatore impedisce di chiamare `drop` su un valore mentre esiste un *borrow* attivo su di esso, esattamente come impedirebbe la creazione di un riferimento mutabile in presenza di riferimenti immutabili attivi. Questo principio tornerà utile quando analizzeremo la prevenzione del use-after-free.

### 2.4.3 Il trasferimento di ownership non deve lasciare accessi non validi

La terza proprietà verificata dal borrow checker riguarda il rapporto tra trasferimento di ownership (detto *move*) e validità degli accessi. È un aspetto molto importante, perché la sicurezza della memoria non dipende soltanto dalla durata dei riferimenti o dalla compatibilità dei prestiti, ma anche dal fatto che, quando una risorsa cambia proprietario, il resto del programma non continui a usarla come se nulla fosse.

In Rust, quando un valore *non-Copy* viene assegnato a una nuova variabile, come abbiamo brevemente visto all'inizio del capitolo, non avviene una semplice duplicazione della risorsa, ma un trasferimento di ownership. Questo significa che la variabile originale smette immediatamente di essere valida. La ragione di questa scelta è chiara dal punto di vista della sicurezza: se il vecchio proprietario potesse continuare a usare il dato dopo il trasferimento, il linguaggio riaprirebbe la porta a situazioni analoghe a quelle che nei linguaggi con gestione manuale della memoria stanno all'origine di use-after-free, double free e altri accessi impropri.

Un esempio molto semplice è il seguente:

```
fn main() {
    let a = vec![1,2,3]; // Vec è nello heap e non è di tipo copy
    let b = a;           // ownership trasferita (move)
    println!("{:?}", a); // ERRORE: a è stato mosso
    println!("{:?}", b); // OK: b è il nuovo proprietario
}
```

Figura 2.10: Uso di una variabile dopo un move.

Qui `Vec<T>` è un vettore a dimensione variabile, allocato nello heap, che non implementa `Copy`. Quando si esegue `let b = a` l'ownership del vettore passa a `b`. Da quel momento `a` non rappresenta più un accesso valido alla risorsa. Il compilatore, quindi, impedisce l'uso successivo di `a`. Questo è esattamente il

significato della terza regola: il trasferimento di ownership non deve lasciare nel programma accessi residui che continuino a trattare la risorsa come se fosse ancora controllata dal vecchio proprietario.

Questo controllo è concettualmente molto importante, perché mostra che il borrow checker non osserva solo i riferimenti in senso stretto, ma la coerenza complessiva del ciclo di vita degli oggetti. Un dato può essere ancora “vivo” in memoria, ma non per questo ogni variabile che un tempo vi era associata rimane autorizzata a usarlo. Dopo un *move*, il problema non è che il valore sia stato distrutto immediatamente; il problema è che il diritto di accesso tramite il vecchio owner non è più valido. Rust formalizza questa idea e la fa rispettare staticamente.

Si può dire, in sintesi, che il borrow checker protegge tre dimensioni della correttezza degli accessi: la *durata*, l'*esclusività* e la *continuità della validità* dopo il cambio di proprietario.

## 2.5 Ownership condivisa: `Rc<T>` e `Arc<T>`

Come scritto, il modello base di Rust prevede un unico owner per ogni valore. In alcuni scenari, tuttavia, questa restrizione è eccessiva: strutture dati come grafi, alberi, o componenti condivisi tra più parti di un programma richiedono che più entità mantengano vivo lo stesso dato contemporaneamente. Rust risolve questo problema con i tipi `Rc<T>` (*Reference Counted*) e `Arc<T>` (*Atomically Reference Counted*), che implementano *ownership condivisa* tramite conteggio dei riferimenti.

```
use std::rc::Rc;

fn main() {
    let a = Rc::new(String::from("dato condiviso"));

    let b = Rc::clone(&a); // incrementa il contatore: ora è 2
    let c = Rc::clone(&a); // contatore: ora è 3

    println!("a = {}", a);
    println!("b = {}", b);
    println!("c = {}", c);

    // Il valore viene deallocato solo quando
    // a, b e c escono tutte dallo scope
}
```

Figura 2.11: Ownership condivisa con `Rc<T>`.

`Rc::clone` non duplica il dato sottostante, ma incrementa un contatore interno. Il valore viene deallocato solo quando il contatore raggiunge zero, cioè quando tutti

i riferimenti che lo possiedono escono dallo scope. `Arc<T>` è la variante *thread-safe*: il contatore viene aggiornato tramite operazioni atomiche, rendendo sicuro l'uso da più thread. Questo meccanismo è completamente *safe* dal punto di vista degli accessi alla memoria: non si crea mai un dangling pointer, perché il dato rimane allocato finché almeno un `Rc` o `Arc` lo riferisce.

## 2.6 Interior mutability: `Cell<T>`, `RefCell<T>` e `Mutex<T>`

Il borrow checker impone a *compile time* che esista o un solo riferimento mutabile o un numero arbitrario di riferimenti immutabili, ma mai entrambi simultaneamente. Questa restrizione è sufficiente per la grande maggioranza dei casi, ma risulta eccessivamente conservativa in alcuni scenari: una cache o un contatore di accessi in una struttura condivisa sono tutti pattern che il borrow checker statico non è in grado di gestire direttamente. Per questi casi Rust offre i tipi di *interior mutability*, che spostano il controllo sull'aliasing dal *compile time* al *runtime*, mantenendo comunque le garanzie di memory safety.

`Cell<T>` è il caso più semplice: permette di leggere e scrivere un valore attraverso un riferimento immutabile, ma funziona esclusivamente con tipi *Copy*. Non espone mai un riferimento diretto al valore interno, il che rende il meccanismo sicuro senza necessità di controlli a runtime.

`RefCell<T>`, a differenza di `Cell`, permette di ottenere riferimenti veri al valore interno, sia immutabili (`borrow()`) che mutabili (`borrow_mut()`), ma verifica dinamicamente che le regole del borrow checker siano rispettate nel momento della chiamata. Se la verifica fallisce – perché esiste già un *borrow* mutabile attivo, o perché si tenta di ottenere un borrow mutabile mentre ne esiste già uno immutabile – il programma non produce comportamento indefinito, ma termina con un *panic*.

```
use std::cell::RefCell;

fn main() {
    let x = RefCell::new(vec![1, 2, 3]);

    let r1 = x.borrow();    // borrow immutabile – ok
    let r2 = x.borrow_mut(); // PANIC a runtime: esiste già un borrow attivo

    println!("{:?}", r1);
    println!("{:?}", r2);
}
```

Figura 2.12: Panic di `RefCell` a runtime.

Questo aspetto è importante per la valutazione delle garanzie di Rust. Il programma sopra compila correttamente ma va in panic a runtime. La memoria non viene mai corrotta: il panic è deterministico, controllato e privo di comportamento indefinito. Giusto per completezza, per evitare il panic basterebbe fare un `drop(r1)` prima di dichiarare `r2`. In contesti multi-thread, `RefCell<T>` non è utilizzabile e l'equivalente *thread-safe* è `Mutex<T>`, che serializza gli accessi tramite un lock acquisito a runtime.

## Capitolo 3

# La protezione degli attacchi: use-after-free, double free

### 3.1 Use-after-free nello heap

In C/C++ è possibile accedere a una risorsa nello heap anche dopo la sua liberazione:

```
int main() {
    int *p = malloc(sizeof(int)); // alloca un intero sull'heap e assegna il puntatore a p
    *p = 42;                      // scrive il valore 42 nella memoria allocata
    int *q = p;                   // copia il puntatore: q punta alla stessa memoria di p
    free(p);                     // libera la memoria heap
    printf("%d\n", *q);           // use-after-free: q dereferenzia memoria già liberata
}
```

Figura 3.1: Use-after-free nello heap in C.

Questo tipo di errore, denominato *use after free* (UAF), può essere sfruttato, come abbiamo visto a lezione, per manipolare le strutture interne dell'*allocator* o per ottenere accessi a memoria arbitraria. In Rust una situazione logicamente equivalente viene bloccata dal compilatore quando questo forza l'applicazione dei principi prima enunciati:

```
fn main() {
    let p = Box::new(42); // alloca un intero nello heap
    let q = &p;           // q prende in prestito p
    drop(p);              // ERRORE: non posso distruggere p mentre è preso in prestito
    println!("{}", q);     // q userebbe un dato che sarebbe stato liberato
}
```

Figura 3.2: UAF nello heap rifiutato da Rust.

Poiché *q* prende in prestito *p*, il compilatore impedisce che *p* venga distrutto mentre il prestito è ancora attivo: l'errore viene segnalato sul `drop(p)`, non sull'accesso successivo tramite *q*. Questo è direttamente il secondo controllo del borrow checker – il divieto di aliasing pericoloso – combinato con il primo: il riferimento *q* non deve avere un lifetime più lungo del valore a cui si riferisce, e il

compilatore lo verifica staticamente prima ancora che l'accesso non valido possa avvenire.

## 3.2 Use-after-free nello stack

Si può in modo molto simile analizzare il caso di un use-after-free anche nello stack. In C un comportamento del tutto lecito e non rilevato dal compilatore è il seguente:

```
#include <stdio.h>

int* foo() {
    int x = 42;      // variabile locale sullo stack
    return &x;       // ritorna indirizzo di memoria che non sarà più valido
}

int main() {
    int *p = foo(); // p riceve un dangling pointer
    printf("%d\n", *p); // comportamento indefinito
}
```

Figura 3.3: UAF nello stack in C.

Nel quale la funzione restituisce un valore che non è più disponibile perché allocato in una porzione dello stack che viene liberata quando la funzione termina. In Rust questa situazione viene impedita dal compilatore:

```
fn foo() -> &i32 {           // tentativo di restituire un riferimento
    let x = 42;              // variabile locale sullo stack
    &x                       // ERRORE: x non vive abbastanza
}

fn main() {
    let p = foo();
    println!("{}", p);
}
```

Figura 3.4: UAF nello stack rifiutato da Rust.

Più precisamente la situazione viene impedita dal compilatore già in fase di analisi della firma della funzione. Vedendo `fn foo() -> &i32` senza alcun parametro di tipo riferimento, il compilatore conclude che non esiste nessuna sorgente esterna da cui il lifetime del riferimento restituito potrebbe provenire – l'unica origine

possibile è una variabile locale, che per definizione non sopravvive alla funzione. È il primo controllo del borrow checker: un riferimento non deve avere un lifetime più lungo del valore a cui si riferisce.

### 3.3 Double free

Una vulnerabilità strettamente correlata è la liberazione multipla della stessa memoria.

```
int main() {
    int *p = malloc(sizeof(int)); // Alloca nello heap memoria per un intero.
    free(p);                      // Libera la memoria allocata.
    free(p);                      // ERRORE: libera di nuovo la stessa memoria (double free).
}
```

Figura 3.5: Double free in C.

Questo comportamento può compromettere le strutture dello *heap allocator* e costituire una base per exploit complessi.

In Rust la stessa situazione non è rappresentabile nel codice *safe*:

```
fn main() {
    let p = Box::new(5); // Alloca nello heap un intero con ownership in p.
    drop(p);             // Libera esplicitamente la memoria posseduta da p.
    drop(p);             // ERRORE: p è già stato consumato da drop.
}
```

Figura 3.6: Double free rifiutato da Rust.

Quando `drop(p)` viene chiamato la prima volta, il valore viene distrutto e l'ownership viene consumata – è il terzo controllo del borrow checker: il trasferimento di ownership non deve lasciare nel programma accessi non più validi. A quel punto `p` non è più una variabile valida, e il compilatore lo sa staticamente. La seconda chiamata `drop(p)` viene quindi bloccata a *compile time*: non perché il compilatore rilevi una doppia liberazione di memoria, ma perché `p` dopo il primo *drop* non esiste più come valore utilizzabile. Il double free viene così reso non esprimibile nel linguaggio, non semplicemente rilevato.

## Capitolo 4

# Spatial safety e attacchi di buffer overflow

Una classe tipica di vulnerabilità è rappresentata dai *buffer overflow*, che derivano da violazioni della *spatial memory safety*, ossia accessi a memoria oltre i limiti dell'oggetto allocato. Nelle slide del corso tale problema viene inizialmente illustrato nel contesto dello stack e successivamente nello heap: nel contesto dello stack, una scrittura oltre i limiti può corrompere il *return address* e il *frame pointer*, aprendo la strada a *code injection*, *return-to-libc* e ROP. Nello heap, la corruzione della memoria adiacente può invece coinvolgere metadati dell'allocatore (come visto a lezione), *function pointer* o oggetti, portando a tecniche di sfruttamento diverse.

Rust implementa tre diversi meccanismi per impedire problemi di spatial memory safety:

- impedisce la deferenziazione di qualsiasi puntatore diretto alla memoria così come qualsiasi aritmetica dei puntatori;
- introduce i *wide pointers*;
- implementa un meccanismo di controllo sui limiti spaziali della memoria usata.

## 4.1 Nessuna deferenziazione né aritmetica dei puntatori

A differenza di quanto avviene in C e C++, in *safe* Rust non è consentito manipolare direttamente puntatori raw. Sebbene il linguaggio permetta la creazione di puntatori raw (`*const T`, `*mut T`), il codice *safe* non ne consente la dereferenziazione né l'uso dell'aritmetica dei puntatori, operazioni che renderebbero possibile la costruzione manuale di indirizzi di memoria arbitrari e quindi l'accesso a regioni al di fuori dello spazio assegnato.

```
fn main() {
    let addr = 0x1000 as *const u32; // raw pointer a indirizzo arbitrario

    // let value = *addr;           // ERRORE: dereferenziazione non consentita in safe Rust

    unsafe {                       // uso possibile solo in unsafe
        let value = *addr;         // dereferenziazione raw pointer
        println!("{}", value);
    }
}
```

Figura 4.1: Raw pointer non dereferenzabile in safe Rust.

In questo esempio viene dichiarato un *raw pointer* ma se si cercasse di dereferenziarlo il compilatore solleverebbe un errore. Tale operazione è ammessa esclusivamente all'interno di blocchi `unsafe`, nei quali il compilatore non è più in grado di garantire automaticamente le proprietà di sicurezza della memoria. Vedremo più avanti questo caso.

## 4.2 Slices e wide pointers

In *safe Rust*, l'accesso e la manipolazione di buffer di memoria avvengono tramite astrazioni tipizzate che rendono esplicita l'estensione della regione di memoria accessibile. Tale informazione può essere nota staticamente, come nel caso degli *array* a dimensione fissa dove il tipo del dato incorpora la sua dimensione, oppure essere mantenuta a runtime nella struttura o nel riferimento stesso, come accade per `Vec<T>` e `String` e per i riferimenti a sequenze contigue come le *slice* (`&[T]`) e le stringhe slice (`&str`), che incorporano la lunghezza direttamente nel riferimento. Una *slice* in Rust è una vista (*view*) su una sequenza contigua di elementi memorizzati in memoria, che permette di accedere a una porzione di dati senza possederli né copiarli.

```
fn main() {
    let data = [10, 20, 30, 40]; // Array di quattro elementi
    let s = &data[1..3];        // Slice che crea una vista sugli elementi con indice 1 e 2

    println!("{:?}", s);        // Output: [20, 30]
}
```

Figura 4.2: Slice e array a dimensione fissa.

In particolare, i riferimenti a tipi dinamicamente dimensionati sono spesso descritti come *wide pointer*, poiché contengono non solo l'indirizzo del primo elemento, ma anche metadati aggiuntivi necessari per utilizzare la memoria in

modo sicuro. Nel caso delle slice, tali metadati corrispondono alla lunghezza della regione accessibile, consentendo alla libreria standard di verificare automaticamente il rispetto dei limiti durante operazioni di accesso o copia dei dati.

Questo modello contribuisce in modo significativo alla prevenzione di accessi fuori dai limiti e, di conseguenza, alla mitigazione delle vulnerabilità di tipo buffer overflow nel codice *safe* e ha conseguenze dirette anche sulle operazioni di copia. In Rust, come abbiamo già discusso, l'operatore di assegnamento non rappresenta una copia generica di memoria tra buffer contigui, come avviene tipicamente nei linguaggi C e C++. L'assegnamento opera invece a livello semantico sul valore, trasferendone l'ownership oppure duplicandolo in base alle proprietà del tipo. Di conseguenza, non è possibile utilizzare l'assegnamento per effettuare copie byte-per-byte tra regioni di memoria arbitrarie. Quando si desidera copiare esplicitamente i dati contenuti in buffer contigui, è quindi necessario utilizzare slice e metodi come `copy_from_slice`, che verificano automaticamente la compatibilità delle dimensioni prima di effettuare la copia. Questo approccio consente alla libreria standard di prevenire scritture fuori dai limiti della memoria allocata, contribuendo in modo sostanziale alla *spatial memory safety*.

```
fn main() {  
    let src = [1, 2, 3, 4];           // Buffer sorgente  
    let mut dst = [0; 4];           // Buffer destinazione di uguale dimensione  
  
    dst.copy_from_slice(&src);       // Copia sicura: controllo automatico della lunghezza  
  
    println!("{:?}", dst);           // Output: [1, 2, 3, 4]  
}
```

Figura 4.3: Copia sicura tra slice con `copy_from_slice`.

## 4.3 Bounds checking statico e dinamico

Sulla base dei meccanismi descritti, Rust garantisce che ogni accesso indicizzato a un array o a una slice avvenga nel rispetto dei limiti della regione di memoria valida. Quando l'indice è noto staticamente e risulta sicuramente fuori intervallo, il compilatore è in grado di rilevare l'errore già in fase di compilazione. Quando invece l'indice dipende da valori determinati a runtime, il compilatore inserisce automaticamente un controllo dei limiti nel codice generato; qualora tale controllo fallisca durante l'esecuzione, il programma termina con un *panic* controllato anziché provocare una corruzione della memoria.

In C, un accesso fuori dai limiti può essere compilato e produrre comportamenti indefiniti del programma:

```
int main() {
    int buf[4];
    buf[5] = 10; /* scrittura fuori dai limiti dell'array */
}
```

Figura 4.4: Accesso fuori dai limiti in C.

Con gli accessi indicizzati in *safe* Rust non esiste una scrittura oltre i limiti senza protezione. Se il compilatore può dimostrare già a *compile-time* che l'indice è certamente fuori range, segnala il problema:

```
fn main() {
    let buf = [1, 2, 3, 4]; // array di quattro elementi
    let x = buf[10];        // ERRORE: indice certamente fuori dai limiti
    println!("{}", x);
}
```

Figura 4.5: Bounds check statico in Rust.

Il codice sopra non viene infatti compilato.

In Rust, l'accesso fuori dai limiti di un array può essere rilevato a *compile time* solo quando l'indice è una costante che il compilatore può valutare staticamente.

```
fn main() {
    let buf = [10, 20, 30, 40]; // array di quattro elementi
    let indici = [0, 1, 2, 10]; // l'ultimo indice è fuori dai limiti

    for i in indici {
        let x = buf[i]; // bounds check a runtime: panic quando i = 10
        println!("{}", x); // non viene raggiunta all'ultima iterazione
    }
}
```

Figura 4.6: Bounds check dinamico con indice a runtime.

In Rust, quando la dimensione di una struttura dati è nota solo a runtime (come nel caso di `Vec<T>`) il compilatore non può verificare i limiti staticamente, ma inserisce automaticamente un controllo a runtime:

```
fn main() {
    let mut buf: Vec<i32> = vec![1, 2, 3, 4]; // vettore con 4 elementi
    let i = 6; // indice fuori dai limiti
    buf[i] = 99; // bounds check a runtime → panic
}
```

Figura 4.7: Bounds check dinamico su `Vec<T>`.

Il compilatore vede che `buf` è di tipo `Vec<i32>` – un tipo la cui lunghezza non fa parte della firma del tipo ma è un valore interno noto solo a runtime – e che `i` è una variabile. Non ha quindi informazioni sufficienti per decidere a *compile time* se l’accesso è valido, e delega il controllo all’esecuzione. Quando il programma raggiunge quella riga, il runtime verifica che `i < buf.len()` e, non essendo così, termina con un *panic* controllato invece di corrompere silenziosamente la memoria come farebbe C.

Anche quando l’array è parte di una struttura più complessa, l’accesso rimane confinato al campo corretto:

```
struct S {           // definizione di una struttura dati
    char buf[4];      // piccolo buffer di 4 byte
    int flag;         // variabile intera memorizzata subito dopo il buffer
};

struct S s;          // dichiarazione di una variabile di tipo struct S

s.buf[6] = 'C';       // accesso fuori dai limiti del buffer: può sovrascrivere flag
```

Figura 4.8: Overflow di struct in C.

In Rust una costruzione analoga non consente di oltrepassare i limiti del campo:

```
struct S {           // definizione di una struttura dati
    buf: [u8; 4],     // array di 4 byte
    flag: i32,        // campo intero memorizzato dopo il buffer
}

let mut s = S {      // creazione di una variabile mutabile di tipo S
    buf: [0; 4],      // inizializzazione dell'array
    flag: 0,          // inizializzazione del campo intero
};

s.buf[6] = 1;        // accesso fuori limite → il controllo dei bounds blocca l'operazione
```

Figura 4.9: Struct in Rust: accesso confinato al campo.

## Capitolo 5

# Riassumendo: Rust e gli attacchi visti a lezione

Vediamo in sintesi come gli attacchi che abbiamo studiato a lezione, basati sul linguaggio C/C++, vengono mitigati o prevenuti in Rust.

| Vulnerabilità           | Come avviene in C                                                | Come Rust la previene                                                                                                   | Effetto in safe Rust                                                                                       |
|-------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Buffer overflow</b>  | Scrittura oltre i limiti di un array o buffer                    | Controllo dei limiti sugli array e tipi dinamici                                                                        | Non causa corruzione della memoria; nel caso peggiore causa <i>panic</i> a runtime, proteggendo la memoria |
| <b>Heap overflow</b>    | Scrittura oltre la memoria allocata                              | API sicure che non espongono aritmetica sui puntatori e garantiscono che ogni accesso resti nei limiti dell'allocazione | Non causa corruzione della memoria; nel caso peggiore causa <i>panic</i> a runtime, proteggendo la memoria |
| <b>Use after free</b>   | Uso di un puntatore dopo <code>free()</code>                     | Principio di ownership e move                                                                                           | Impedito a <i>compile time</i>                                                                             |
| <b>Double free</b>      | <code>free()</code> chiamata due volte                           | Principio di ownership (un solo proprietario)                                                                           | Impedito a <i>compile time</i>                                                                             |
| <b>Dangling pointer</b> | Uso di un puntatore a variabile fuori dallo scope                | Borrow checker e lifetimes                                                                                              | Impedito a <i>compile time</i>                                                                             |
| <b>ROP</b>              | Possibili dopo la corruzione della memoria                       | Eliminazione delle vulnerabilità di base                                                                                | Non ha effetto perché i suoi presupposti sono impediti a <i>compile time</i>                               |
| <b>Data race</b>        | Due thread accedono alla stessa memoria con almeno una scrittura | <i>Aliasing XOR Mutability</i>                                                                                          | Impedito a <i>compile time</i>                                                                             |

Tabella 5.1: Sintesi degli attacchi e della loro mitigazione in Rust.

Chiaramente l'uso di codice `unsafe` apre una porta alle vulnerabilità elencate, come discusso.

## Capitolo 6

# I limiti di Rust

### 6.1 Unsafe Rust

`unsafe` definisce un blocco di istruzioni all'interno del quale il compilatore non controlla più l'utilizzo della memoria in termini di sicurezza.

Nella pratica, `unsafe` è necessario in almeno tre scenari tipici:

1. **Interoperabilità con C e C++**, dove Rust deve dialogare con codice che non rispetta necessariamente le stesse garanzie. Rust supporta questo tipo di integrazione tramite il meccanismo chiamato *FFI* (*Foreign Function Interface*), che permette a un programma Rust di chiamare funzioni implementate in altri linguaggi o di esporre funzioni Rust a programmi esterni.

```
use std::ffi::c_char;

unsafe extern "C" {
    fn puts(s: *const c_char) -> i32; // funzione C legacy
}

fn main() {
    let msg = b"Hello from legacy C\0"; // stringa C terminata da '\0'

    unsafe { // chiamata FFI
        puts(msg.as_ptr() as *const c_char);
    }
}
```

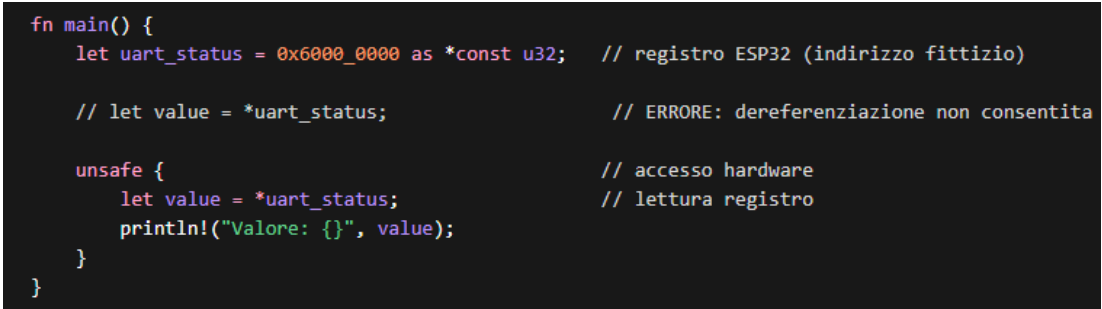
Figura 6.1: Dichiarazione FFI in Rust.

Il compilatore Rust non può verificare le garanzie di sicurezza del codice esterno: non può sapere, ad esempio, se un puntatore passato a una funzione C sia valido o se la funzione stessa rispetti le invarianti richieste da Rust. Per questo motivo le chiamate FFI devono essere effettuate all'interno di un blocco `unsafe`.

2. **Implementazione di astrazioni low-level efficienti**, come allocatori, contenitori o primitive di runtime. Molte delle astrazioni fondamentali del *type system* di Rust, come `Box`, `Vec` o `Arc`, sono implementate internamente utiliz-

zando codice `unsafe`. Questo è necessario perché tali strutture devono gestire direttamente la memoria *heap* tramite *raw pointer* e operazioni di allocazione che il compilatore non può verificare automaticamente. L'interfaccia pubblica di queste strutture è tuttavia progettata in modo da essere completamente *safe*: il codice `unsafe` rimane confinato all'interno dell'implementazione e mantiene una serie di invarianti che garantiscono la correttezza delle operazioni. Con il termine *invarianti* si indicano le proprietà interne che una struttura dati deve mantenere sempre vere per garantire la sicurezza della memoria. Ad esempio, nel caso di `Vec<T>`, l'implementazione deve assicurare che il puntatore interno sia valido, che la lunghezza non superi la capacità e che gli elementi accessibili siano inizializzati. Il codice `unsafe` è utilizzato proprio per manipolare direttamente la memoria mantenendo queste invarianti; l'API pubblica della struttura, invece, rimane completamente *safe*. Si noti infine che la ricerca recente sulla sicurezza di Rust si concentra proprio su questi confini tra codice *safe* e *unsafe*, sviluppando strumenti di analisi statica e tecniche di *fuzzing* per individuare eventuali violazioni di tali invarianti. In questo senso, il modello di sicurezza di Rust non elimina completamente la possibilità di errori, ma ne riduce fortemente la superficie concentrandoli in piccole porzioni di codice esplicitamente marcate come `unsafe`.

3. **Codice di sistema**, come kernel, driver hardware o componenti di basso livello.

A screenshot of a code editor with a dark background and light-colored text. The code is in Rust and shows a function `main` that attempts to access hardware memory. It defines a constant `uart_status` as `0x6000_0000` and uses it in an `unsafe` block to dereference a pointer and print the value. Comments in Italian explain the hardware access and the error of dereferencing a non-consented pointer.

```
fn main() {  
    let uart_status = 0x6000_0000 as *const u32; // registro ESP32 (indirizzo fittizio)  
  
    // let value = *uart_status; // ERRORE: dereferenziazione non consentita  
  
    unsafe { // accesso hardware  
        let value = *uart_status; // lettura registro  
        println!("Valore: {}", value);  
    }  
}
```

Figura 6.2: Accesso a memoria mappata in unsafe Rust.

In questi contesti è spesso necessario accedere direttamente a indirizzi di memoria specifici, registri hardware o regioni di memoria mappate tramite *memory-mapped I/O*. Anche in questo caso il compilatore non può verificare la validità dell'indirizzo utilizzato [2, 9].

In sostanza, una volta entrati in `unsafe`, il compilatore non può più proteggere completamente il sistema da un uso scorretto della memoria. Questo esempio

fa vedere come una errata architettura del codice `unsafe` possa reintrodurre un problema di use-after-free che avevamo già risolto in Rust:

```
fn main() {
    let b = Box::new(42);
    let p = Box::into_raw(b); // ottengo raw pointer

    unsafe {
        drop(Box::from_raw(p)); // libero la memoria
        println!("{}", *p);    // use-after-free → comportamento indefinito
    }
}
```

Figura 6.3: UAF reintrodotta tramite `unsafe`.

La *Rustonomicon* – ovvero il manuale ufficiale di Rust dedicato alla programmazione `unsafe` – insiste proprio su questo punto: il programmatore deve mantenere manualmente la robustezza delle astrazioni `unsafe` [9].

Nel complesso, il modello di sicurezza di Rust può essere interpretato come una strategia di *riduzione strutturale del rischio*, piuttosto che come una garanzia assoluta di sicurezza. L'uso di codice `unsafe` e l'interoperabilità con componenti esterne introducono una *Trusted Computing Base* (TCB) la cui correttezza diventa fondamentale per preservare le proprietà di memory safety. Di conseguenza, Rust non elimina la possibilità di vulnerabilità, ma ne concentra l'origine in porzioni di codice più limitate e meglio identificabili. Inoltre, errori di progettazione, bug logici o violazioni di protocolli applicativi possono compromettere la sicurezza del sistema anche in assenza di corruzione della memoria.

## 6.2 Uso di `Rc` e `RefCell` e riferimenti circolari

Il modello di ownership di Rust garantisce che ogni valore venga deallocato esattamente una volta, quando il suo proprietario esce dallo scope. Quando si utilizza `Rc<T>`, tuttavia, questa garanzia si indebolisce: la memoria viene liberata non quando il proprietario esce dallo scope, ma quando il contatore di riferimenti raggiunge zero. In presenza di riferimenti circolari il contatore non raggiunge mai zero, e la memoria non viene mai liberata.

Il caso tipico è quello di due nodi che si referenziano a vicenda. Per costruire questo pattern è necessario combinare `Rc<T>` con `RefCell<T>`: `Rc<T>` permette la proprietà condivisa tra più variabili, mentre `RefCell<T>` permette di modificare il

contenuto anche quando il valore è condiviso, spostando il controllo del *borrow* a runtime.

```
use std::rc::Rc;           // Rc: ownership condivisa
use std::cell::RefCell;    // RefCell: mutabilità interna

struct Nodo { next: Option<Rc<RefCell<Nodo>>> } // ogni nodo può puntare a un altro nodo

fn main() {
    let a = Rc::new(RefCell::new(Nodo { next: None })); // crea nodo A (count = 1)
    let b = Rc::new(RefCell::new(Nodo { next: None })); // crea nodo B (count = 1)

    a.borrow_mut().next = Some(Rc::clone(&b)); // A → B (count B = 2)
    b.borrow_mut().next = Some(Rc::clone(&a)); // B → A (count A = 2)

} // fine scope: count A = 1 e count B = 1 → ciclo ⇒ memory leak logico
```

Figura 6.4: Riferimenti circolari con Rc e RefCell.

Quando `a` e `b` escono dallo scope, le variabili locali vengono rimosse e i rispettivi contatori scendono da 2 a 1 – ma non a zero, perché ognuno dei due nodi tiene ancora in vita l’altro tramite il campo `next`. Il risultato è che i due oggetti rimangono allocati in memoria per tutta la durata del programma, senza che nessuna variabile possa più raggiungerli. Questo è un caso in cui *safe* Rust non garantisce *memory liveness* – ovvero che tutta la memoria allocata venga prima o poi liberata. Non si tratta di una violazione di memory safety: nessun accesso non valido avviene, nessun comportamento indefinito si produce. Ma la memoria viene sprecata, e in un programma che crea molti cicli di questo tipo il problema può diventare significativo.

La soluzione offerta da Rust è `Weak<T>`: un riferimento “debole” che non incrementa il contatore forte. La convenzione è che uno dei due lati del ciclo utilizzi `Weak<T>` invece di `Rc<T>`, in modo che quando tutti i riferimenti forti cadono a zero la memoria venga liberata, anche se esistono ancora riferimenti deboli. Questo approccio introduce però un nuovo onere per il programmatore: un riferimento `Weak<T>` può diventare non valido in qualsiasi momento, se tutti i riferimenti forti al dato vengono rimossi. Per accedere al valore è necessario chiamare `Weak::upgrade()`, che restituisce un `Option<Rc<T>>` – `Some` se il dato esiste ancora, `None` se è già stato deallocato. Rust impedisce quindi di accedere direttamente a un `Weak` senza verificarne la validità, mantenendo la memory safety, ma la gestione corretta della logica – decidere quali lati del ciclo debbano essere forti e quali deboli – rimane responsabilità del programmatore. Una scelta sbagliata può portare a deallocazioni premature o, al contrario, a riproporre il problema dei cicli.

## 6.3 Il compilatore `rustc` e le opzioni per la gestione della memoria

Il compilatore `rustc` supporta a pieno tutti i meccanismi di sicurezza che abbiamo visto a lezione:

| Exploit mitigation                                                  | Supported | Enabled by default                                                                           | Since                                                                     |
|---------------------------------------------------------------------|-----------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Position-independent executable                                     | Yes       | Yes                                                                                          | 0.12.0 (2014-10-09)                                                       |
| Integer overflow checks                                             | Yes       | (enabled when debug assertions are enabled, and disabled when debug assertions are disabled) | 1.1.0 (2015-06-25)                                                        |
| Non-executable memory regions                                       | Yes       | Yes                                                                                          | 1.8.0 (2016-04-14)                                                        |
| Stack clashing protection                                           | Yes       | Yes                                                                                          | 1.20.0 (2017-08-31)                                                       |
| Read-only relocations and immediate binding                         | Yes       | Yes                                                                                          | 1.21.0 (2017-10-12)                                                       |
| Heap corruption protection                                          | Yes       | Yes                                                                                          | 1.32.0 (2019-01-17) (via operating system default or specified allocator) |
| Stack smashing protection                                           | Yes       | No, <code>-Z stack-protector</code>                                                          | Nightly                                                                   |
| Forward-edge control flow protection                                | Yes       | No, <code>-Z sanitizer=cfi</code>                                                            | Nightly                                                                   |
| Backward-edge control flow protection (e.g., shadow and safe stack) | Yes       | No, <code>-Z sanitizer=shadow-call-stack,safestack</code>                                    | Nightly                                                                   |

Figura 6.5: Opzioni del compilatore `rustc` per la memoria (*exploit mitigation table*).

## Capitolo 7

# Prestazioni

### 7.1 Rust non usa un garbage collector

Molti linguaggi moderni ottengono memory safety con un *garbage collector* (GC). Java, C# e Go, ad esempio, delegano al runtime il compito di individuare gli oggetti non più raggiungibili e liberarli automaticamente. Questo approccio riduce drasticamente il rischio di use-after-free e double free, ma introduce anche costi strutturali.

Per un linguaggio *general purpose* questo compromesso è spesso accettabile; per un linguaggio di sistema, invece, non sempre lo è. Kernel, librerie crittografiche, componenti di un sistema operativo, driver, *hypervisor* e runtime *embedded* hanno spesso vincoli di latenza e di interoperabilità che rendono un *garbage collector* poco attraente. Rust sceglie quindi una strada diversa: garantire sicurezza della memoria senza GC, facendo leva su regole statiche di ownership e borrowing [1, 3].

Questa scelta è coerente con il modo in cui Rust si presenta ufficialmente. Il *Rust Book* insiste sul fatto che Rust vuole combinare “*high-level ergonomics and low-level control*” e fornire “*zero-cost abstractions*”, cioè astrazioni ad alto livello che non impongano *overhead* di *runtime* rispetto a un’implementazione manuale equivalente [1, 4].

### 7.2 Confronto con C e C++

Non è che Rust sia sempre più veloce di C o C++, ma il suo modello di sicurezza non impone un *overhead* strutturale paragonabile a quello di molte altre strategie di memory safety.

Un benchmark comparativo del 2022, “*Is Rust C++-fast?*”, misura algoritmi e strutture dati d’uso comune e conclude che la performance complessiva di Rust è simile a quella di C++, con solo uno svantaggio medio modesto, e con alcuni casi in cui Rust risulta persino leggermente più veloce [5]. Un lavoro più recente sui *NAS Parallel Benchmarks*, pubblicato nel 2025, riporta che Rust è mediamente circa l’1,23 % più lento di Fortran ma il 5,59 % più veloce della versione C++ inclusa nel confronto [6]. Infine, uno studio del 2024 sulla traduzione di programmi C in

Rust osserva che le traduzioni *safe* prodotte dagli utenti mantengono prestazioni comparabili al C [7].

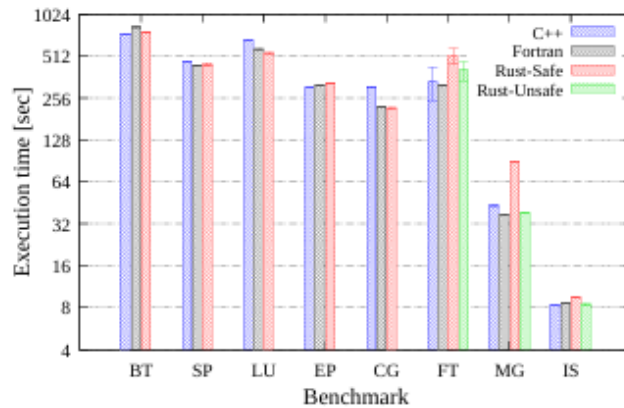


Figura 7.1: Confronto prestazioni Rust vs Fortran/C++ sui NAS Parallel Benchmarks [6].

Infine il fatto che aziende come Google e Microsoft, come vedremo a breve, abbiano deciso di usare Rust in elementi critici dei loro sistemi è una ulteriore conferma della sostanziale convenienza nell'utilizzo di questo linguaggio.

Questi risultati vanno letti con cautela: nessun *benchmark* singolo autorizza affermazioni assolute, e il comportamento cambia con il *workload*, l'ottimizzazione e la qualità dell'implementazione. Tuttavia, nel loro insieme, supportano bene la posizione secondo cui Rust è un linguaggio plausibile per rimpiazzare C/C++ in componenti sensibili, proprio perché offre memory safety senza il tipo di costo sistematico che si assocerebbe a un linguaggio con *garbage collector* [4, 5, 6, 7].

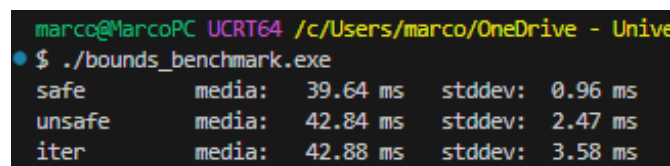
## 7.3 Il costo del bounds checking: un esperimento

Un aspetto specifico delle prestazioni di Rust che merita un approfondimento diretto riguarda il *bounds checking*: il controllo automatico che il compilatore inserisce ad ogni accesso a un array o a uno *slice* per verificare che l'indice sia nei limiti. Questo meccanismo è uno dei pilastri della *spatial safety* descritta nel Capitolo 4, e in questo lavoro abbiamo voluto approfondirlo per capire meglio le implicazioni in termini di performance.

Ispirandosi al lavoro di ricerca di Sergey Davidoff [14, 15] è stato creato un codice che confronta tre varianti della stessa operazione – la somma di tutti gli elementi di un array di 100 milioni di interi `u64` – creando tre varianti che fanno la stessa cosa ma in modi diversi:

- **standard**: si usa l'accesso standard `data[i]`, che in Rust include un *bounds check*. Il compilatore deve analizzare da solo se può eliminarlo.
- **unsafe**: si usa codice `unsafe` per accedere direttamente alla struttura dati bypassando i controlli di Rust. È il modo che intuitivamente sembrerebbe il più veloce.
- **nativo**: si usa un iteratore, cedendo al compilatore il controllo completo su come scorrere gli elementi. È la forma nativa di Rust.

Il risultato è che le tre varianti sono praticamente identiche in termini di tempo! Ciò dimostra che il compilatore è abbastanza intelligente da eliminare i *bounds check* ridondanti anche nella variante *safe*, arrivando allo stesso codice macchina di `unsafe`.



```
marco@MarcoPC UCRT64 /c/Users/marco/OneDrive - Unive
$ ./bounds_benchmark.exe
safe      media:  39.64 ms  stddev:  0.96 ms
unsafe    media:  42.84 ms  stddev:  2.47 ms
iter      media:  42.88 ms  stddev:  3.58 ms
```

Figura 7.2: Output del benchmark `bounds_benchmark`: le tre varianti (*standard*, *unsafe*, *nativo*) ottengono tempi sostanzialmente identici.

## Capitolo 8

# Il database Juliet e il suo utilizzo nello studio della memory safety

Per la parte sperimentale di questo lavoro è stato utilizzato il database *Juliet Test Suite* [13], una collezione di programmi di test sviluppata dal *National Institute of Standards and Technology* (NIST) con l'obiettivo di fornire casi sistematici e riproducibili di vulnerabilità software. Il progetto Juliet nasce infatti come strumento di supporto alla ricerca nel campo della sicurezza informatica e della verifica automatica del codice, consentendo di analizzare in modo controllato il comportamento dei programmi in presenza di specifiche debolezze note. Ogni esempio contenuto nel database è associato a una o più *Common Weakness Enumeration* (CWE), ovvero classificazioni standard delle vulnerabilità, e rappresenta un caso concreto in cui una determinata debolezza può manifestarsi.

Nel contesto delle tecniche di protezione del software, il database Juliet è particolarmente utile perché permette di osservare come errori tipici della gestione manuale della memoria si traducano in comportamenti indefiniti, crash o condizioni sfruttabili per attacchi. Inoltre, la struttura modulare della suite consente di confrontare direttamente implementazioni vulnerabili e implementazioni corrette, facilitando sia lo studio teorico sia la sperimentazione pratica. Per questo motivo Juliet è spesso utilizzato nella ricerca accademica, nello sviluppo di strumenti di analisi statica e dinamica, e nella valutazione dell'efficacia di nuovi linguaggi o paradigmi di programmazione orientati alla sicurezza.

## 8.1 Metodologia sperimentale

L'obiettivo della sperimentazione non è dimostrare genericamente che Rust sia un linguaggio sicuro, ma analizzare in modo sistematico come cambia la possibilità di esprimere specifiche classi di vulnerabilità di *memory corruption* quando si passa da C a *safe Rust*.

Sono state selezionate cinque categorie rappresentative di vulnerabilità direttamente correlate alle proprietà di temporal safety e spatial safety, in particolare:

- Double free

- Use-after-free
- Dangling pointer (*return of stack variable address*)
- Stack-based buffer overflow
- Heap-based buffer overflow

Queste categorie sono state scelte perché rappresentano in modo diretto e complementare le principali violazioni delle proprietà di temporal e spatial safety studiate nella parte teorica ed a lezione, consentendo di coprire le condizioni strutturali alla base degli attacchi di *memory corruption*.

Per ciascun caso di studio sono state svolte le seguenti attività:

1. Analisi del comportamento del programma originale presente in Juliet Test Suite e scritto in C, con particolare attenzione al pattern di gestione della memoria responsabile della vulnerabilità.
2. Traduzione concettuale del codice in *safe Rust*, mantenendo il più possibile invariata la logica applicativa.
3. Osservazione del comportamento del compilatore Rust e del sistema a runtime, distinguendo tra:
  - vulnerabilità bloccate a *compile time*;
  - vulnerabilità rilevate tramite controlli a runtime.
4. Confronto sistematico tra le modalità con cui i due linguaggi gestiscono l'errore.

Questo approccio consente di evidenziare in quale fase del ciclo di vita del programma (compilazione o esecuzione) intervengano le garanzie di sicurezza offerte da Rust.

## 8.2 Risultati osservati

L'analisi dei casi di studio evidenzia una regolarità sistemica nel modo in cui Rust previene le vulnerabilità di *memory corruption*. In tutti i casi analizzati emerge una correlazione tra la natura della vulnerabilità e la fase in cui Rust ne impedisce l'esecuzione: le violazioni di temporal safety vengono sistematicamente intercettate a *compile-time*, mentre le violazioni di spatial safety sono gestite tramite una combinazione di verifica statica e controlli *runtime*.

Le vulnerabilità riconducibili a violazioni di temporal safety, come double free, use-after-free e dangling pointer, risultano non esprimibili in *safe Rust*. In questi casi, il compilatore impedisce la compilazione del programma poiché il modello di ownership e borrowing, insieme ai concetti di *lifetime* e *scope*, non consente di:

- utilizzare un valore dopo il trasferimento di ownership;

- accedere a memoria già deallocata;
- creare riferimenti che sopravvivono al valore referenziato.

Di conseguenza, la vulnerabilità viene eliminata prima dell'esecuzione del programma, spostando la prevenzione dal *runtime* al *compile time*.

Diversa è la situazione per alcune violazioni di spatial safety, come il buffer overflow su array o vettori. In questi casi, *safe Rust* può consentire la compilazione del programma quando l'errore non è determinabile staticamente, introducendo automaticamente controlli sui limiti (*bounds checking*). Quando un accesso fuori dai limiti viene rilevato a runtime, il programma termina in modo controllato tramite *panic*, evitando la corruzione silenziosa della memoria.

Nel complesso, i risultati evidenziano che:

- Rust trasforma le vulnerabilità di *temporal memory-unsafe* in errori di compilazione;
- nei casi di *spatial memory-unsafe* analizzati ciò non è sempre possibile (ad esempio quando l'indice della struttura dati è associato ad una variabile il cui valore è noto solo a *run time*), e Rust introduce meccanismi di rilevazione a runtime che impediscono l'esecuzione di exploit basati sulla corruzione della memoria.

Qui sotto i risultati della sperimentazione:

| Vulnerabilità                                     | Sp./Temp. | Compila? | Panic? | Motivo                                                                                                                              |
|---------------------------------------------------|-----------|----------|--------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>CWE 121:</b><br>stack-based buffer<br>overflow | Spatial   | Sì       | Sì     | Il metodo <code>copy_from_slice</code> viene bloccato a <i>run time</i> dal <i>bounds checker</i> e il programma va in <i>panic</i> |
| <b>CWE 122:</b><br>heap-based buffer<br>overflow  | Spatial   | Sì       | Sì     | Il metodo <code>copy_from_slice</code> viene bloccato a <i>run time</i> dal <i>bounds checker</i> e il programma va in <i>panic</i> |
| <b>CWE 415:</b> double<br>free                    | Temporal  | No       | –      | La seconda <code>drop(p)</code> viene bloccata dal compilatore perché <code>p</code> è inutilizzabile dopo il <i>move</i>           |
| <b>CWE 416:</b><br>use-after-free                 | Temporal  | No       | –      | L'uso della variabile viene bloccato perché la variabile è stata rimossa ( <i>move</i> )                                            |
| <b>CWE 562:</b> return of<br>stack variable       | Temporal  | No       | –      | Il compilatore impedisce di restituire una variabile locale                                                                         |

Tabella 8.1: Risultati sperimentali sulle cinque CWE selezionate.

Il contributo sperimentale di questo lavoro consiste nel fornire un confronto strutturato tra classi di vulnerabilità *memory-unsafe* e meccanismi di prevenzione offerti da Rust, evidenziando:

- quali vulnerabilità diventano non esprimibili;
- quali vengono rilevate a runtime;
- come cambia il modello di gestione degli errori rispetto al linguaggio C.

L'analisi mostra come l'adozione di Rust modifichi profondamente lo spazio delle vulnerabilità possibili, riducendo drasticamente le condizioni necessarie per l'esecuzione di attacchi di *memory corruption*. Più avanti vedremo come in Google siano stati trovati dei numeri per sostanziare questo cambiamento.

Va osservato infine che i risultati si riferiscono a un insieme rappresentativo ma limitato di vulnerabilità di *memory corruption* e non implicano che Rust prevenga automaticamente errori logici o vulnerabilità introdotte tramite codice `unsafe` o interfacce FFI.

### 8.3 Insegnamenti dalla parte sperimentale

Dall'analisi dei casi di studio emergono alcune osservazioni:

- **Eliminazione delle violazioni di temporal safety a compile-time.** Le vulnerabilità come double free, use-after-free e dangling pointer risultano non esprimibili in *safe* Rust. Il modello di ownership e borrowing trasforma tali *pattern* in errori di compilazione, impedendo l'esecuzione di programmi *memory-unsafe*.
- **Rilevazione controllata delle violazioni di spatial safety.** Le violazioni di spatial safety vengono gestite in modo differenziato a seconda della possibilità di determinarle staticamente. Quando l'accesso fuori dai limiti può essere dedotto dal compilatore (ad esempio nel caso di indici costanti o di dimensioni note a *compile-time*), il programma viene rifiutato in fase di compilazione. Negli altri casi, *safe* Rust introduce controlli automatici sui limiti che intercettano l'errore durante l'esecuzione, causando una terminazione controllata del programma (*panic*) e prevenendo la corruzione silenziosa della memoria tipica dei programmi vulnerabili in C.

Nel complesso, la sperimentazione evidenzia come l'adozione di un linguaggio *memory safe* riduca drasticamente le condizioni strutturali necessarie per l'esecuzione di attacchi di *memory corruption*.

## Capitolo 9

# Uso di Rust fuori dall'ambito accademico: Android, Microsoft e Linux

### 9.1 Android

Negli ultimi anni Google [10, 11] ha analizzato in modo sistematico il ruolo delle vulnerabilità legate a problematiche di memory safety nella sicurezza della piattaforma Android e ha introdotto una strategia per ridurle alla radice. Storicamente queste vulnerabilità hanno rappresentato la principale fonte di problemi di sicurezza nel sistema: nel 2019 circa il 76 % delle vulnerabilità della piattaforma Android era causato da errori di gestione della memoria, come buffer overflow o use-after-free, tipicamente associati all'uso di linguaggi come C e C++. Per affrontare il problema, Google ha iniziato a promuovere lo sviluppo del nuovo codice in linguaggi memory-safe, tra cui Rust, Java, Kotlin e Go, evitando per quanto possibile l'introduzione di nuovo codice in linguaggi non memory-safe. Questo approccio, definito *Safe Coding*, si basa sull'idea che prevenire le vulnerabilità a livello di linguaggio sia più efficace che tentare di individuarle e correggerle dopo la loro introduzione.

I risultati riportati da Google mostrano un cambiamento significativo. Nell'arco di circa sei anni, la percentuale di vulnerabilità legate a problemi di memory safety nella piattaforma Android è scesa dal 76 % nel 2019 al 24 % nel 2024. In termini assoluti, alcune analisi indicano che il numero di vulnerabilità di questo tipo è diminuito da circa 223 casi nel 2019 a meno di 50 nel 2024 (cfr. Figura 9.1). Questo risultato è stato ottenuto senza riscrivere completamente il codice esistente: gran parte della riduzione deriva semplicemente dal fatto che il nuovo codice viene scritto in linguaggi memory-safe, mentre il codice legacy continua a essere progressivamente mantenuto e corretto.

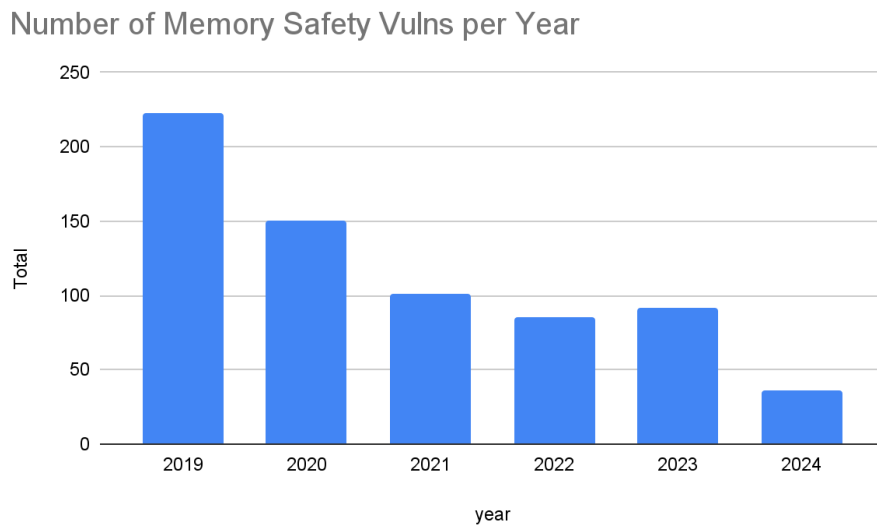


Figura 9.1: Andamento delle vulnerabilità di memory safety su Android (fonte: Google Security Blog).

Un ulteriore risultato riportato dal team di sicurezza Android riguarda l'impatto specifico di Rust. Secondo i dati interni di Google, il codice scritto in Rust mostra una densità di vulnerabilità di memory safety circa 1000 volte inferiore rispetto al codice C/C++ utilizzato nella piattaforma. Nel complesso, questi risultati suggeriscono che l'adozione progressiva di linguaggi memory-safe può ridurre in modo significativo intere classi di vulnerabilità nei sistemi complessi, senza richiedere una riscrittura completa dell'infrastruttura esistente.

## 9.2 Microsoft e il porting della libreria crittografica SymCrypt

Anche il caso Microsoft è particolarmente interessante perché riguarda una libreria crittografica fondamentale. Microsoft Research ha annunciato la riscrittura in Rust di parti di *SymCrypt*, la libreria *open-source* che alimenta funzionalità crittografiche usate in Windows, Azure Linux, Xbox e altre piattaforme [12].

Il collegamento con le *crypto API* di Windows è importante. La documentazione Microsoft sulle API di sicurezza di sistema indica che i servizi crittografici nativi del sistema sono alimentati da SymCrypt. Questo significa che la scelta di portare algoritmi e componenti di SymCrypt in Rust non è un semplice esperimento, ma incide sullo strato crittografico usato da API reali del sistema operativo.

Nel post del 2025 Microsoft spiega che la motivazione è duplice: aumentare la memory safety e rendere più praticabile la verifica formale, senza rinunciare a

prestazioni elevate. Il progetto parte da una implementazione in Rust di ML-KEM e mira a integrare codice Rust nel flusso di distribuzione di SymCrypt, mantenendo compatibilità con il resto dell’ecosistema [11]. Questo è un esempio molto forte, perché mostra che Rust non viene adottato solo in componenti applicativi, ma anche in una libreria crittografica di base, dove la combinazione tra performance, correttezza e sicurezza è particolarmente critica.

## 9.3 Rust nel kernel Linux

Negli ultimi anni il linguaggio Rust è stato progressivamente introdotto anche nello sviluppo del kernel Linux, tradizionalmente scritto quasi esclusivamente in C e *assembly*. In particolare, il kernel Linux ha introdotto il supporto ufficiale al linguaggio Rust a partire dalla versione 6.1, dopo l’approvazione di una *pull request* da parte di Linus Torvalds nel 2022 (“Rust for Linux”). Questa integrazione iniziale ha riguardato soprattutto l’infrastruttura di base necessaria per consentire agli sviluppatori di sperimentare la scrittura di driver e moduli kernel in Rust, mantenendo comunque una portata limitata per evitare rischi di regressioni nel codice esistente. L’inserimento del supporto è stato infatti confermato da diverse fonti tecniche che riportano come il codice iniziale per Rust sia stato integrato nel ramo principale del kernel durante il ciclo di sviluppo della versione 6.1.

L’interesse verso Rust nel kernel è legato principalmente alla possibilità di ridurre errori legati alla gestione manuale della memoria. In questo contesto, studi sul progetto *Rust-for-Linux* mostrano che moduli del kernel riscritti in Rust possono ridurre completamente alcune classi di vulnerabilità di memory safety (da circa 10 bug a 0 nei prototipi analizzati). Questo risultato è coerente con l’obiettivo generale del progetto *Rust-for-Linux*, nato proprio per sfruttare le garanzie di sicurezza del linguaggio nella scrittura di driver e componenti a basso livello, diminuendo il rischio di errori come use-after-free o buffer overflow.

Nel complesso, l’adozione graduale di Rust nel kernel Linux rappresenta un cambiamento significativo nel paradigma della programmazione di sistema. Pur mantenendo il linguaggio C come base storica del progetto, la possibilità di utilizzare Rust in nuovi moduli consente di sperimentare un modello di sviluppo orientato alla memory safety, con l’obiettivo di ridurre nel lungo periodo la superficie di attacco derivante da vulnerabilità legate alla gestione della memoria.

## Capitolo 10

# Conclusioni

In questo lavoro è stato analizzato il contributo del linguaggio Rust alla prevenzione delle vulnerabilità di *memory corruption* attraverso un duplice approccio teorico e sperimentale.

Dal punto di vista teorico, sono stati studiati i principali meccanismi del linguaggio – ownership, borrowing, lifetimes e controlli sui limiti degli accessi alla memoria – mettendoli in relazione con le classi di vulnerabilità analizzate nel corso, tipicamente osservabili nei programmi scritti in C.

Dal punto di vista sperimentale, è stata condotta una comparazione mirata tra programmi vulnerabili in C tratti dal database *Juliet* e porting concettuali degli stessi in *safe Rust*, appositamente per questo lavoro. Sono state selezionate cinque vulnerabilità rappresentative – double free, use-after-free, dangling pointer, stack-based buffer overflow, heap-based buffer overflow – al fine di coprire in modo diretto le principali violazioni delle proprietà di temporal e spatial safety viste a lezione. Per ciascun caso sono stati sviluppati esempi di codice Rust volti a verificare se e in quale fase il linguaggio impedisse l'espressione della vulnerabilità e, in caso positivo, se lo facesse a *compile time* o a *run time*. I risultati mostrano che le tre vulnerabilità riconducibili alla temporal safety analizzate (double free, use-after-free e dangling pointer) risultano non esprimibili in *safe Rust*, in quanto il compilatore impedisce la compilazione del programma. La vulnerabilità riconducibile alla spatial safety considerata (stack-based e heap-based buffer overflow) può invece compilare quando l'errore non è determinabile staticamente, ma la vulnerabilità specifica dell'esempio viene intercettata tramite controlli automatici a runtime che impediscono la corruzione silenziosa della memoria.

Nel complesso, l'analisi dimostra che Rust modifica in modo strutturale le condizioni che rendono possibili gli attacchi basati sulla corruzione della memoria, anticipando la prevenzione di specifiche classi di vulnerabilità alla fase di compilazione oppure trasformandole in errori di esecuzione controllati. Allo stesso tempo, lo studio evidenzia che tali garanzie non sono assolute: l'uso di codice *unsafe*, l'interoperabilità con componenti esterne e la presenza di errori logici o di progettazione rimangono fattori critici che possono compromettere la sicurezza

---

complessiva del sistema.

L'analisi sperimentale ha inoltre mostrato che l'introduzione di meccanismi di memory safety non comporta necessariamente un degrado significativo delle prestazioni rispetto alle implementazioni equivalenti in C, evidenziando come Rust possa rappresentare un compromesso efficace tra sicurezza ed efficienza nei contesti di programmazione di sistema.

Infine, l'analisi di casi industriali reali – come l'adozione di Rust nella piattaforma Android, nella libreria crittografica SymCrypt di Microsoft e in componenti del kernel Linux – mostra come questo cambiamento di paradigma non sia soltanto teorico, ma già concretamente in atto nello sviluppo di software critico. La scelta di grandi aziende tecnologiche di utilizzare Rust per la realizzazione di componenti fondamentali suggerisce infatti una progressiva transizione verso modelli di programmazione memory-safe destinati a diventare sempre più centrali nella progettazione dei sistemi software del futuro.

Il contributo principale di questo lavoro consiste quindi nel mettere in relazione in modo sistematico i meccanismi linguistici di Rust con esempi concreti di vulnerabilità e con evidenze sperimentali mirate, includendo anche valutazioni sulle prestazioni, e nel mostrare in modo circoscritto e verificabile come il linguaggio possa ridurre il rischio di *memory corruption* senza compromettere i requisiti di efficienza tipici della programmazione di sistema.

# Bibliografia

- [1] Steve Klabnik, Carol Nichols et al., *The Rust Programming Language (Rust Book)*. <https://doc.rust-lang.org/book/>.
- [2] *Rust Standard Library Documentation*. <https://doc.rust-lang.org/std/>.
- [3] *Rust Foundation*. <https://foundation.rust-lang.org/>.
- [4] *The Rust Programming Language – sito ufficiale*. <https://www.rust-lang.org/>.
- [5] Nikolay Ivanov, *Is Rust C++-fast? Benchmarking System Languages on Everyday Routines*, 2022.
- [6] Eduardo M. Martins et al., *NPB-Rust: NAS Parallel Benchmarks in Rust*, 2025.
- [7] Ruishi Li et al., *Translating C To Rust: Lessons from a User Study*, NDSS 2025 / arXiv 2024.
- [8] Bohan Qin et al., *Understanding and Detecting Real-World Safety Issues in Rust*, IEEE TSE 2024.
- [9] *The Rustonomicon*; Rust Standard Library Documentation on `unsafe`. <https://doc.rust-lang.org/nomicon/>.
- [10] Google Security Blog, *Eliminating Memory Safety Vulnerabilities at the Source*, 2024.
- [11] Google Security Blog, *Rust in Android: Move Fast and Fix Things*, 2025.
- [12] Microsoft Research Blog, *Rewriting SymCrypt in Rust to Modernize Microsoft’s Cryptographic Library*, 2025.
- [13] National Institute of Standards and Technology (NIST), *Juliet Test Suite*, 2022.
- [14] Serghey Davidoff, *How to Avoid Bounds Checks in Rust (Without Unsafe)*, 2023.
- [15] Serghey Davidoff, *Shnatsel/bounds-check-cookbook: Recipes for Avoiding Bounds Checks in Rust, Without Unsafe*, 2023.